SPECSHIP · FOR PRODUCT OWNERS & DELIVERY LEADS

Build the right thing — and prove it's done.

How SpecShip turns AI coding assistants from fast-but-forgetful helpers into a delivery system you can trust.

THE NEW REALITY

AI assistants now write much of the code. That moved the bottleneck.

Typing is no longer slow

An assistant such as Claude Code can draft a feature in minutes.

Understanding still is

Before it can change code safely, the assistant has to work out how today's system fits together — every single session.

Rework is the new cost

Fast code that misses the intent, or quietly breaks something nearby, comes back as bugs, review rounds and re-work.

> **The question is no longer “how fast can we write code?” but “how fast can we ship the *right* code — and know it?”**

THE PRODUCT OWNER'S PROBLEM

Your intent leaks out between the ticket and the code.

- **“Done” is a status, not a fact.** A ticket says done; nothing shows which acceptance criteria are actually proven.
- **Requirements drift silently.** Code changes next month and nobody notices it no longer matches what you agreed.
- **Traceability is manual.** “Which code implements this requirement — and is it tested?” takes a developer's afternoon to answer.
- **AI speed amplifies all three.** More code, faster, with the same blind spots.

What you actually want

For every requirement, at a glance:

- Is it built?
- Is it **proven** by a passing test?
- Has anything changed since we agreed it?

SpecShip answers those three questions for every requirement written as a spec — and flags the moment the code moves away from them.

THE DEVELOPER'S PROBLEM

Every session, the assistant re-reads the codebase from scratch.

Ask “what happens when a user checks out?” and an assistant without a map searches files, opens them, searches again, opens more — many steps (on a large codebase, dozens) — just to find where to look.

- Slow: time spent searching before any real work.
- Fragile: it can miss a connection between files and break something it never saw.
- Wasteful: the same discovery is repeated tomorrow, and by every teammate.

Analogy

A brilliant new hire with amnesia

Imagine a contractor who is excellent — but forgets the building's floor plan every morning and has to walk every corridor again before fixing a single tap.

SpecShip hands them the floor plan.

Every search the assistant repeats is time your developer waits for — and a connection it can still miss.

WHAT SPECSHIP IS

One tool, three layers: a map, a contract, and proof.

Layer 1 · The map

Knows your code

A precise, always-current index of how every part of the system connects. The assistant asks the map instead of re-reading files.

Lives in: Inside Claude Code

Layer 2 · The contract

Knows your intent

Requirements written as plain-language specs with acceptance criteria — each one linked to the exact code that delivers it.

Lives in: Spec files in your repo — mirrored to JIRA

Layer 3 · The proof

Knows it's done

Tests prove each criterion; quality gates can block what isn't proven; drift alerts fire when code moves away from the agreement.

Lives in: Dashboard, JIRA and CI

Plugs into Claude Code, runs entirely on your machine, and can be adopted one layer at a time.

LAYER 1

The map

How SpecShip stops the assistant re-reading your codebase — and why that makes it faster and safer.

LAYER 1 · THE MAP

A GPS for your codebase, instead of asking for directions at every corner.

- **Built automatically** from the code itself — every function, every "who calls whom", and the web routes of common frameworks. Nothing hand-maintained.
- **Exact, not guessed.** The map is derived mechanically from the code's structure, never summarised by an AI — so it doesn't hallucinate.
- **Always current.** It refreshes itself within seconds of a code change.
- **20 programming languages**, across web, mobile and back-end stacks.

In practice

Question: "How does a failed payment reach the customer email?"

Without the map: search → open → search → open … until the trail is found (or missed).

With the map: one lookup returns the whole path, with the relevant code, in one go.

Bonus: "what would changing this break?" is answered before anyone edits a line.

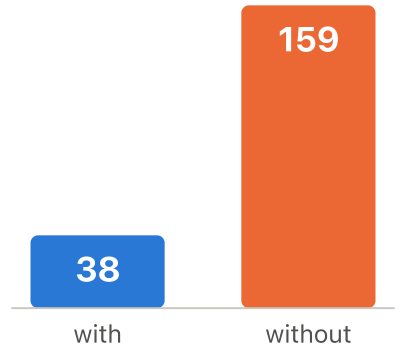
The result: the assistant spends its time on your change, not on finding its way around.

MEASURED, NOT PROMISED

Same assistant, same question — with and without SpecShip.

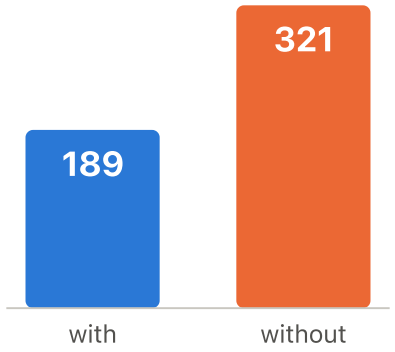
With SpecShip Without SpecShip

Files opened
total across 37 projects



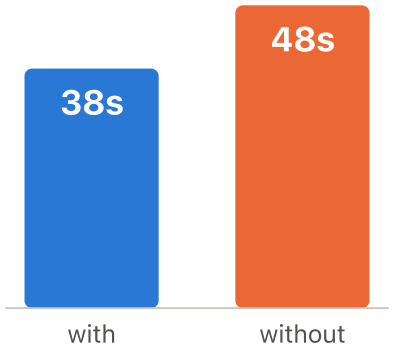
−76%

Steps taken
total across 37 projects



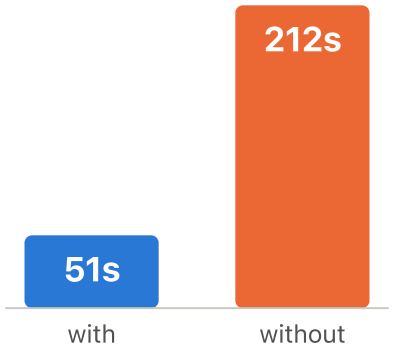
−41%

Time to answer
average per question



−21%

Best single case
biggest gain · ~2,000-file app



4.2× faster (one project)

Source: SpecShip A/B benchmark, 2026-05-24, run on v0.9.4 (this deck describes v0.27) — 37 open-source projects in 16 languages, one code-flow question each, same AI model and prompt; SpecShip was the only difference. One run per project, so treat small differences as noise. Gains grow with codebase size; on tiny projects the two are even, and SpecShip is occasionally a few seconds slower. Cost per question was slightly higher (about 12%) on these short questions — the win is speed and fewer steps.

LAYER 2

The contract

Your requirements become living specs — written in plain language, owned by the team, and wired to the code.

LAYER 2 · THE CONTRACT

A spec is a requirement you can check — not a document that gathers dust.

REQ-AUTH-001 **Failed logins MUST be rate-limited**

The login page rejects more than 5 failed attempts per minute from the same place.

Acceptance

A1 · The 6th failed attempt within 60 s is refused and says when to retry.

A2 · A successful login resets the count.

implemented by → login code

proven by → rate-limit test

Simplified for the slide — real specs point at exact code and test names.

- **Plain language.** Readable by a product owner; precise enough for an engineer and an AI assistant.
- **Every requirement and criterion has a permanent ID** (REQ-AUTH-001, A1, A2) — so it can be tracked, linked and reported on.
- **Linked both ways:** to the code that delivers it and to the test that proves it.
- **Lives with the code**, so it is versioned and reviewed like everything else — and can be mirrored to JIRA.

LAYER 2 · WRITING SPECS

You describe the feature. SpecShip helps turn it into a solid spec.

1 · IDEA

Capture

Jot down an idea or a brief. It's tracked as an idea, so nothing gets lost.

›

2 · DRAFT

Draft from the code

The assistant drafts the spec using the map, so it points at the real parts of the system.

›

3 · GAPS

Ask what's missing

A few questions on edge cases, failure paths and ownership — only the ones not already answered.

›

4 · CHECK

Quality review

Every criterion must be testable; a spec checker flags missing IDs, criteria and test plans.

Your business language

Domain terms (“settlement”, “active member”) are captured as confirmed facts the AI then uses consistently.

Learns your conventions

Corrected the assistant the same way in two separate sessions? SpecShip proposes it as a team rule for the next session.

Already in JIRA?

Specs publish as stories and sub-tasks; edits made in JIRA are proposed back to the spec for approval.

LAYER 3

The proof

From “the ticket says done” to “here is the passing test for every acceptance criterion.”

LAYER 3 · THE PROOF

Every requirement carries an honest, live status.

STATUS	WHAT IT MEANS FOR YOU
● Implemented	Code exists that claims to deliver this requirement. **Not yet proven.**
● Verified ✓	A linked test **actually ran and passed**. This is the only status that means “done”.
● Drifted ⚠	The code (or the spec) changed after it was linked. Someone should re-check it still matches.
● Broken ✕	Verification ran and **failed**. The requirement is not met today.
● Orphaned ?	The code it pointed to was removed or renamed. The link needs re-attaching.

“Verified” cannot be self-awarded — without real test evidence, SpecShip refuses to promote it.

LAYER 3 · DRIFT

Drift: the smoke alarm for “it used to match the requirement.”

When the code behind a requirement changes, SpecShip notices — automatically, within seconds — and flags the requirement as **drifted**.

- No one has to remember which requirements a change touches.
- Drifted items land in a **review queue** with one-click repair actions.
- A requirement is only cleared when someone re-checks it — or a test re-proves it.

Example

Monday: “Refunds over $500 need manager approval” — verified ✓

Thursday: a developer refactors the refund code for performance.

Seconds later: the requirement shows **Drifted** ⚠ and appears in the review queue — before it reaches customers.

Drift turns “did anything change?” from a meeting into a notification.

LAYER 3 · QUALITY GATES

Quality gates catch the shortcuts fast AI code is prone to.

Proof required

A requirement without a passing test isn't counted as done. Coverage reports show which criteria are proven — and which aren't.

No silencing warnings

If a change quietly switches off a quality check instead of fixing the problem, the gate spots it and stops the change.

Architecture rules

Your team's structural rules (“the UI never talks to the database directly”) are checked automatically.

Health signals

Oversized files, tangled dependencies and other warning signs surface before they become expensive.

Tests that test

Tests are written to prove each criterion, not just to pass — and, optionally, your existing test-quality tools report whether they would catch real bugs.

Only what changed

The gate can focus on just the requirements a change touches — fast enough to run on every commit.

Gates start as advisory and are switched to blocking one at a time, when the team is ready.

PUTTING IT TOGETHER

From requirement to proven code

The built-in delivery workflow — from an approved plan to a proven change.

THE IMPLEMENT WORKFLOW

One command takes a spec to reviewed, tested code.

1 · PLAN
Plan
Reads the spec and the map; proposes how to build it.

›

YOU APPROVE
Plan review
Approve it — or reject with feedback and it revises.

›

2 · BUILD
Build & self-check
Writes the code in an isolated copy, then runs the quality gates.

›

3 · PROVE
Prove & critique
Runs the tests; an independent reviewer grades each criterion satisfied, partial or unaddressed.

›

YOU APPROVE
Final review
See the real change and the critic's verdict before anything is kept.

You make **two decisions**; the assistant does the building, testing and self-review in between.

Tests and quality gates compute pass/fail; a separate reviewer pass critiques the result — and nothing is kept without your approval.

A WORKED EXAMPLE

The life of one requirement, as a product owner sees it.

DAY 1

Ticket → spec

“Refunds over $500 need manager approval” becomes REQ-REFUND-004 with three acceptance criteria — and a JIRA story, if JIRA is connected.

DAY 2

Plan approved

The developer runs the workflow; you or the lead approve the plan. The assistant builds it in an isolated copy.

DAY 3

Verified ✓

Tests pass for all three criteria. The status turns **Verified** and a progress comment lands on the JIRA story.

WEEK 6

Drifted ⚠

A refactor touches the refund code. The requirement flips to **Drifted** and shows in the review queue.

WEEK 6

Re-proven

The tests are re-run, the requirement returns to **Verified** — and the history shows why.

Card colour: **blue** = in progress · **green** = proven · **amber** = needs a re-check

At every point, “is it done, and still done?” has a one-glance answer — no meeting required.

WHERE THE SPEED COMES FROM

Compound gains across delivery — one stage measured, four to measure with you.

STAGE	WHAT USUALLY COSTS TIME	WHAT SPECSHIP CHANGES	EVIDENCE
Find the code	Searching and re-reading files	One lookup on the map	Measured
Rework	Bugs and drift found weeks later	Gates and drift alerts catch them early	Not yet measured
Clarify intent	Back-and-forth on what “done” means	Testable criteria agreed up front	Not yet measured
Avoid breakage	Unknown connections break	Impact known before editing	Not yet measured
Review	Reconstructing the “why”	Change linked to its requirement + reviewer verdict	Not yet measured

Why 5× is a credible target

With AI, typing is nearly free. Delivery time is now dominated by understanding, alignment, review and rework — exactly the stages SpecShip targets.

What is proven today

Finding the code: 21% faster on average, up to 4.2× on a large project. 5× end-to-end is our **target, not a measurement** — the pilot measures it on your team.

WHERE YOU SEE IT

The surfaces

Developers work inside their assistant; product owners see the same truth in a dashboard and in JIRA.

THE SURFACES

One source of truth, shown wherever each role works.

SURFACE	WHO	WHAT IT'S FOR
Inside Claude Code	Developers	The assistant uses the map automatically; a handful of simple commands cover the daily flow.
Dashboard	POs, leads, devs	Traceability, the drift queue, workflow runs with approve/reject buttons, and AI-session analytics. Runs locally in the browser.
JIRA	POs, QA, delivery	Specs as stories and sub-tasks, progress comments, sprint coverage reports, and regression test packs for QA.
Command line & CI	Engineering	The same checks run on every commit or pull request, so the gate holds for the whole team.
Gemini CLI	Developers (opt-in)	The map and spec tools are also available in Google's Gemini CLI (slash commands and automatic hooks remain Claude Code only).

THE DASHBOARD · TRACEABILITY

The product owner's window: what's built, what's proven, what moved.

Traceability · 4 of 38 requirements — Illustrative example

REQUIREMENT	CRITERIA	TESTS	STATUS
REQ-REFUND-004 · Manager approval over $500	3 / 3	3	Verified
REQ-AUTH-001 · Rate-limit failed logins	2 / 2	2	Drifted · 2d
REQ-CART-012 · Save cart across devices	1 / 3	1	Implemented
REQ-PAY-007 · Retry failed card payments	0 / 2	2	Broken

- **One row per requirement:** its code, its tests, criteria met, verdict, and how long it has been drifted.
- **Specs funnel:** ideas → specs → built → verified.
- **Drift queue** with repair buttons.
- **Honest numbers:** every figure is measured or clearly marked as an estimate — none are invented.

Ideas	Specs	Built	Verified ✓
12	38	31	24

Specs funnel — illustrative numbers.

WHAT CHANGES FOR EACH ROLE

Everyone gets the same truth, framed for their job.

ROLE	BEFORE	WITH SPECSHIP
Product owner	“Is it done?” means asking around	A live status per requirement and criterion, with the proof attached
Developer	Searching before every change	The assistant knows the codebase; impact is known before editing
QA	Test plans rebuilt from tickets by hand	Regression packs generated from acceptance criteria, results recorded back
Tech lead	Architecture rules enforced by memory	Rules, health and drift checked on every change
Delivery lead	Sprint progress from status fields	Sprint coverage joined to real verification state

What doesn't change: your tracker, your repository, your review process. SpecShip connects them.

GETTING STARTED

Adopt it one rung at a time — climb when the last one has paid off.

RUNG 1 · DAY ONE

The map

Install once; the assistant stops re-reading the code. No process change.

›

RUNG 2

Health & impact

See risky areas and what a change would affect. Still no specs needed.

›

RUNG 3

Specs & proof

Write specs for new work; prove criteria with tests; turn on gates.

›

RUNG 4

Team traceability

Dashboard and JIRA give product and delivery a live view.

The pilot — how we measure the gain on your team

One team · one feature area · two sprints, compared with the previous two sprints.

We report: **cycle time**, **review rounds**, **escaped bugs** and the **% of requirements verified**. SpecShip supplies the verification data; cycle time and bugs come from your own tracker.

Rung 1 needs no process change — start there.

QUESTIONS WE HEAR

Straight answers.

Does our code go to a third party?

Not through SpecShip — its core runs entirely on your machine: no network calls, no telemetry. (Your AI assistant has its own terms.)

Can we trust what the map says?

It comes from the code's structure, not an AI summary — and where it can't be sure, it stays silent rather than guess.

What does writing specs cost us?

A spec is a short plain-language requirement with testable criteria. The assistant drafts it from your description and asks only about gaps; you review it.

Do we have to write specs for everything?

No. The map works with zero specs. Add specs where proof matters most.

Will the AI change things on its own?

No. Plans, final changes, learned rules and business facts all wait for a person's approval, and the delivery workflow builds in an isolated copy. (Day-to-day edits follow your assistant's own permission settings.)

Does it replace JIRA or our process?

No. It connects requirements to code and proof, and mirrors into the JIRA you already use. Integrations are off until switched on.

Which AI assistant — and what does it cost?

Built for Claude Code (Gemini CLI opt-in). SpecShip itself is free and open source (MIT); your assistant subscription is unchanged. Setup: two commands and a restart (plus a one-time index per project).

How soon do we see value?

The map helps from the first session after install. Specs and gates add value feature by feature, as you adopt them.

IN ONE SENTENCE

SpecShip gives your AI assistant a map of the code, a contract for each requirement, and proof that it's met.

Find it faster. Prove it's done. Know when it drifts.